

Friedrich-Alexander-Universität Erlangen-Nürnberg

**Lehrstuhl für Multimediakommunikation und
Signalverarbeitung**

Prof. Dr.-Ing. André Kaup

Master Thesis

**Uncertainty-aware planning using
generative models for autonomous driving**

Jaivardhan Chandak

November 2024

Supervisors: Prof. Dr. Vasileios Belagiannis
Michele Di Vita

Declaration

I confirm that I have written this thesis unaided and without using sources other than those listed and that this thesis has never been submitted to another examination authority and accepted as part of an examination achievement, neither in this form nor in a similar form. All content that was taken from a third party either verbatim or in substance has been acknowledged as such.

Erlangen 19.12.24

Place, Date

Jai

Jaivardhan Chandak
Erlangen

Contents

Abstract	III
Symbols and Notations	V
Abbreviations and Acronyms	VII
1 Introduction	1
1.1 Thesis Structure	2
2 Background	3
2.1 Autonomous Driving	3
2.2 Deep Learning	3
2.2.1 Artificial Neural Networks	3
2.2.2 Stochastic Gradient Descent	4
2.3 Deep Learning Models	5
2.3.1 Convolutional Neural Networks	5
2.3.2 Recurrent Neural Networks	6
2.3.3 Autoencoders	7
2.4 Model Uncertainty	8
2.5 Generative Models	9
2.5.1 Variational Autoencoders	9
3 Methodology	11
3.1 CARLA Simulator	11
3.1.1 CARLA	11
3.1.2 CARNOVEL	12
3.2 Dataset	12
3.2.1 CARLA Dataset	12
3.2.2 Data Sample Breakdown	12
3.3 Performance Metrics	13
3.3.1 Average Displacement Error	13
3.3.2 Final Displacement Error	14

3.3.3	CARNOVEL Metrics	14
4	Model Architecture	17
4.1	Feature Extractor	18
4.1.1	MobileNetV2 Breakdown	19
4.2	Generative Model	19
4.3	Predictor Network	21
4.3.1	GRU Decoder	21
4.3.2	Estimator	22
5	Training	23
5.1	Training the Model	23
5.1.1	Negative Log Likelihood Loss	23
5.1.2	Kullback Leibler Divergence Loss	24
5.2	Further Optimisation	25
5.2.1	Prior	25
5.2.2	Goal Likelihood	25
5.2.3	Posterior	25
6	Results	27
6.1	Training Evaluation	27
6.2	CARNOVEL	29
6.2.1	Abnormal Turns	30
6.2.2	Hills	30
6.2.3	RoundABOUTs	30
6.3	Discussion	31
7	Conclusion	33
A	First Appendix	35
	List of Figures	37
	List of Tables	39
	Bibliography	41

Abstract

This thesis explores the application of Variational Autoencoders (VAEs) to enhance the robustness and adaptability of autonomous driving systems that account for uncertainty in its decision-making process. The proposed approach involves training a VAE to model the distribution of a latent vector obtained by passing driving scenarios through an encoder. Sampling from this distribution generates new representations of the latent vector. Taking an average of these samples allows the agent to work with a more generalised latent vector and make more robust decisions, particularly in uncertain and novel environments.

The experimental results demonstrate the effectiveness of the proposed approach when compared with similarly trained models, highlighting the potential of generative models to improve the safety and reliability of autonomous driving systems. The VAEAgent was tested on the CARNOVEL benchmark, where it was evaluated on scenes unseen to it from training. The VAEAgent scored reasonably in the abnormal turns task, matched the best-performing method in the hills tasks and was dominantly clear of its competitors in the roundabout task.

Symbols and Notations

$\leftarrow$	Assignment
$\alpha \nabla$	Gradient operator with respect to θ
$E_\phi(\cdot)$	Encoder network parameterised by ϕ
$D_\theta(\cdot)$	Decoder network parameterised by θ
$p(\mathbf{z})$	Probability distribution of a random variable $\mathbf{z}$
$z \sim p(\mathbf{z})$	Sample z drawn from $p(\mathbf{z})$
$p(\mathbf{z} = z)$	Probability of $\mathbf{z}$ being some value z
$\min_x f(x)$	The value of x that minimises $f(x)$
β	Hyperparameter to scale the Kullback-Leibler loss term

Abbreviations and Acronyms

In alphabetical order.

AD Autonomous Driving

ADE Average displacement error

AE Autoencoder

API Application programming interface

CIL Conditional Imitative Learning

CNN Convolutional neural networks

DIM Deep Imitative Model

DL Deep learning

DNN Deep neural networks

FDE Final displacement error

GRU Gated recurrent unit

KL Kullback-Leibler

ML Machine learning

MLP Multilayer Perceptron

MSE Mean squared error

MVN Multivariate normal

NLL Negative log likelihood

OOD Out-of-distribution

RIP Robust Imitative Planning

RNN Recurrent neural networks

SGD Stochastic gradient descent

VAE Variational Autoencoder

Chapter 1

Introduction

Autonomous Driving (AD) has made significant strides thanks to advancements in deep learning. Neural network systems can process sensor data from multiple sources in real-time and make accurate decisions to steer and control the vehicle [1]. Models are trained on mountains of driving data and can reach performance levels that rival or even surpass human drivers [2]. Nevertheless, these models are inherently limited by the data they are trained on. What happens when these models are confronted with unforeseen scenarios or atypical driving conditions?

A model will have less confidence in its decisions when seeing something novel to it [3]. The ability of an autonomous vehicle to adapt to diverse and challenging driving scenarios is crucial for ensuring safety and reliability. Therefore, agents need to be aware of the uncertainty in their decisions.

Generative models offer a promising avenue for enhancing the versatility and adaptability of AD systems. By learning the underlying distribution of driving scenarios [4], these models can generate diverse and realistic trajectories, even in situations that the model has not explicitly encountered during training. This proactive approach can mitigate the risks associated with unexpected events and enhance the overall robustness of the autonomous driving system.

While generative models have shown promise in various applications, their potential in autonomous driving, specifically in trajectory planning, remains largely unexplored [5]. This thesis delves into the application of generative models to generate diverse and safe trajectories for autonomous vehicles, addressing the challenges posed by uncertainty and new environments.

1.1 Thesis Structure

This thesis is structured in 5 sections. A brief description of the contents of each section is described here.

- Chapter 1 - Introduction

The topic of the thesis and the motivation behind exploring it are introduced and discussed here.

- Chapter 2 - Background

The foundation knowledge of the topics discussed in this thesis are covered here.

- Chapter 3 - Methodology

The methodology used in this thesis. The data sources and evaluation metrics on how the model will be tested

- Chapter 4 - Architecture

An in-depth breakdown of the model. A look into the different objectives of the model and a justification of the selected design to meet those objectives.

- Chapter 5 - Results

The results of the conducted experiments and a discussion about its findings.

- Chapter 6 - Conclusion

A summary of the findings and conclusions drawn from the project, as well as areas for improvement and further research.

Chapter 2

Background

2.1 Autonomous Driving

The once science-fictional fantasy of driverless vehicles is rapidly evolving into reality, thanks to advancements in Deep learning (DL). AD technology is made possible by a combination of intricate sensors, data processing, and decision-making algorithms. There are six levels of autonomy that AD can be categorised into [6]. Level 0 is without any machine assists and Level 5 is driving without any human input. When discussing AD in the rest of the thesis, we will be assuming the vehicle is operating at Level 5.

Driving simulators have emerged as indispensable tools in the development and testing of AD systems. These virtual environments replicate real-world driving conditions, allowing engineers to rigorously evaluate the performance of autonomous vehicles in a controlled and safe setting [7]. By exposing autonomous vehicles to a wide range of scenarios, including adverse weather conditions, traffic congestion, and unexpected events, simulators enable the identification and mitigation of potential failures and limitations. Furthermore, simulators provide a cost-effective and efficient platform for training and refining autonomous driving algorithms, accelerating the development process and reducing the risks associated with real-world testing.

2.2 Deep Learning

Advancements in DL have seen great breakthroughs in domains such as pattern recognition, and natural language processing amongst many others [8]. In this section, we will briefly introduce DL.

2.2.1 Artificial Neural Networks

Inspired by the neuron in the human brain, an artificial neuron is the fundamental building block from which complex Deep neural networks (DNN) are constructed [9]. It receives inputs, processes them, and produces an output. The inputs are multiplied

by a weight and summed together. The weighted sum is passed through an activation function, introducing non-linearity into the network. Arranging the neurons in layers, and connecting the layers gives us the Multilayer Perceptron (MLP) model.

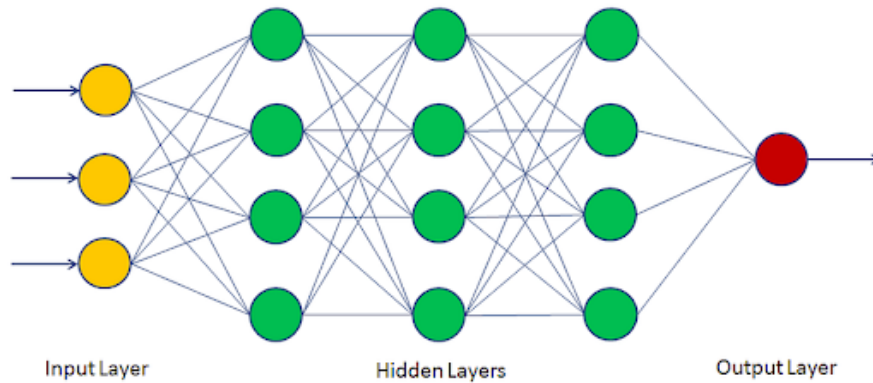


Figure 2.1: An MLP model

The first layer of neurons handles the inputs, while the final layer gives us an output. The remaining layers are the hidden layers [10].

2.2.2 Stochastic Gradient Descent

There exists a sequence of neurons with particular weights and biases that can approximate any function as stated by the Universal Approximation Theorem [9]. Methods are needed to discover the parameter values that can help the DNN approximate the desired function.

Stochastic gradient descent (SGD) is a popular optimisation technique used in Machine learning (ML). By evaluating a model's output by a loss function, such as the mean squared error, SGD iteratively updates parameters to minimise the loss function [11].

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L$$

Each parameter θ is updated iteratively in SGD. The gradient of the loss function L with respect to θ indicates the direction of the steepest ascent. By subtracting the parameter value by this gradient scaled by α , which controls the step size of the update, we move towards the minimum of the loss function.

The gradient $\nabla_{\theta} L$ is calculated using the chain rule, which allows us to propagate the error signal from the output layer all the way back to the input layer [12]. This

process, known as backpropagation, enables the network to adjust its weights and biases in a way that minimises the overall loss.

2.3 Deep Learning Models

Beyond the foundational MLP architecture, specialised artificial neural networks are tailored to specific data types and applications. These architectures, such as Convolutional neural networks (CNN) and Recurrent neural networks (RNN), offer significant advantages in terms of efficiency and performance for their respective use cases [13]. Thus, it is important to discuss them.

2.3.1 Convolutional Neural Networks

CNNs are designed to operate on grid-like data structures such as images and video. They excel at recognising patterns and features within spatial data by exploiting the inherent spatial correlations between pixels [14].

A key component of CNNs is the convolutional layer. This layer applies a set of filters, or kernels, to the input data. Each filter slides across the input, performing element-wise multiplications and summing the results to produce a feature map. By using multiple filters, CNNs can extract a variety of features, such as edges, corners, and textures [15].

Pooling layers are employed to reduce the dimensionality of the feature maps. These layers downsample the feature maps by applying a pooling operation, such as max pooling where only the maximum value from a local region of the feature map is kept and the rest are discarded. Pooling layers reduce computations and redundant information [16].

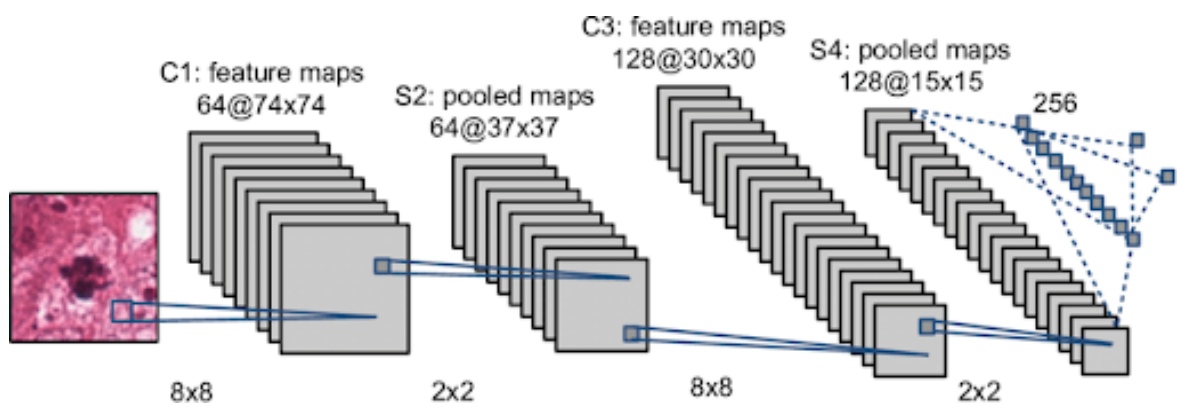


Figure 2.2: CNN model example [17]

2.3.2 Recurrent Neural Networks

Sequential data, such as time series, natural language, and audio signals, is characterized by the temporal dependence between data points. In contrast to static data, where each data point is independent, sequential data requires models that can capture and process information from previous inputs.

RNNs are a class of neural networks specifically designed to handle sequential data. Unlike traditional feedforward neural networks, RNNs have a "memory" component, allowing them to access and process information from previous time steps. This memory, often referred to as the hidden state, allows RNNs to learn long-term dependencies and make predictions based on the entire sequence rather than just the current input [18].

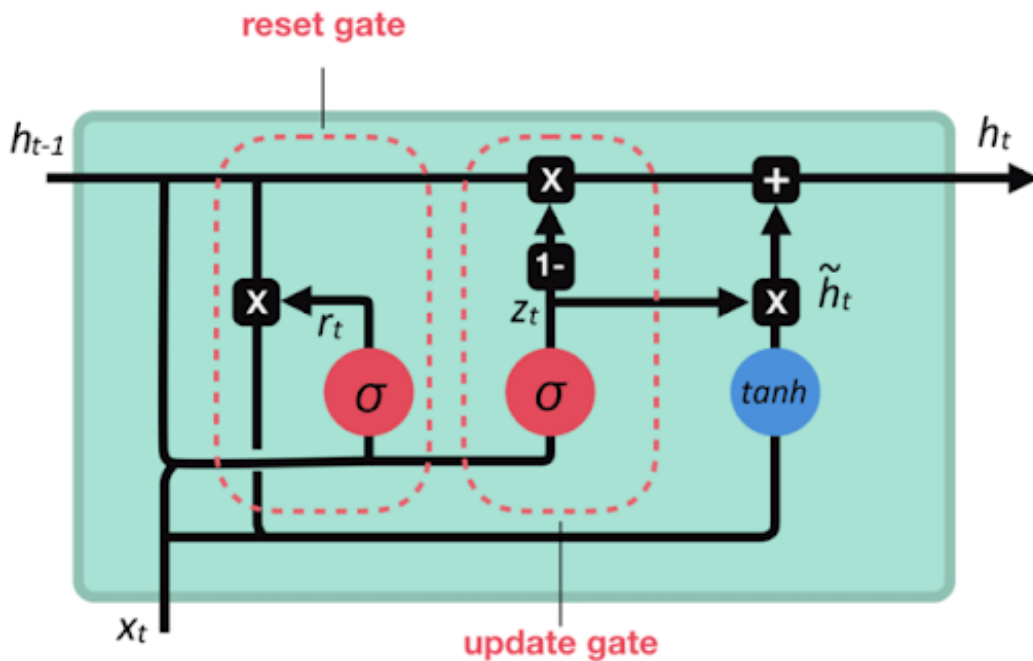


Figure 2.3: GRU model [19]

The vanishing gradient problem is a common challenge in RNN training. Gradients can become exponentially smaller as they propagate backwards through the network, making it difficult for the network to learn long-term dependencies. Gated recurrent unit (GRU) control the flow of information through the network by using two gates. The update gate decides how much of the current input needs to be passed to the future and the reset gate determines how much past information to forget. By selectively updating and forgetting information, GRUs can learn long-term dependencies more

effectively than traditional RNNs and are unaffected by the vanishing gradient issue [20].

2.3.3 Autoencoders

Dimensionality reduction is when data from a higher-dimensional is transformed so it can be represented in a lower-dimensional space called the latent space. When dealing with a large number of data points, scaling the data down can reduce complexity and speed up calculations [21].

Autoencoder (AE) are DL models composed of two parts, an encoder ($E_\phi(\cdot)$) and a decoder ($D_\theta(\cdot)$). The encoder learns how to map some input data $\mathbf{x}^{(i)}$ to a latent space data point $z^{(i)}$. This new representation of data is passed to the decoder, where it can be used for various tasks, the most common one being reconstructing the input $\mathbf{x}'^{(i)}$ [22].

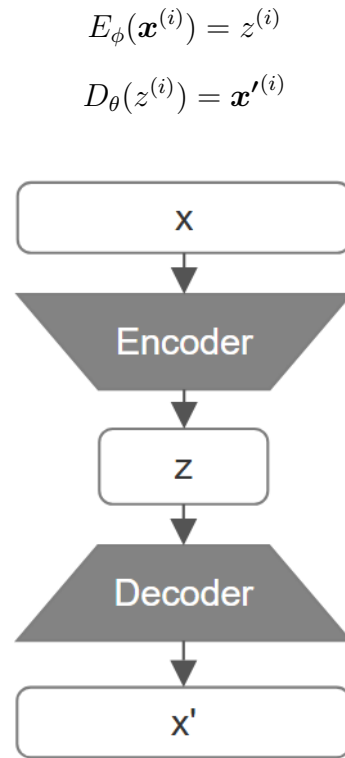


Figure 2.4: An Autoencoder (AE) model

AEs are trained according to their task. In the case of reconstruction, the loss function is the Mean squared error (MSE) of the original input and the AE model output.

$$L_{\text{AE}}(\phi, \theta) = \frac{1}{N} \sum_{i=1}^N \left(\mathbf{x}^{(i)} - D_{\theta}(E_{\phi}(\mathbf{x}^{(i)})) \right)^2$$

2.4 Model Uncertainty

Uncertainty in deep learning models, particularly in the context of autonomous driving, can arise from various sources, including data quality or model complexity. Understanding and mitigating these uncertainties is crucial for the development of safe and reliable autonomous systems. These factors can be labelled as either aleatoric or epistemic uncertainty.

Aleatoric uncertainty stems from inherent randomness in the data itself. This type of uncertainty is irreducible and arises from factors such as measurement errors. In AD, faulty or noisy sensors can lead to uncertainty in object detection and localisation. While we cannot eliminate aleatoric uncertainty, we can account for it by using probabilistic models and incorporating uncertainty estimates into decision-making processes [23].

Epistemic uncertainty reflects the model’s lack of knowledge or confidence in its predictions. This type of uncertainty can be reduced by improving the model’s training data, architecture, or training process. For example, training a model on a larger and more diverse dataset can help to reduce epistemic uncertainty [24].

The capabilities of models reduce dramatically when exposed to environments unseen to them. Autonomous vehicles can regularly encounter situations novel to the data samples on which they were trained. For example, a car trained solely on American roads will likely perform worse when driving on European roads, where roundabouts are more prevalent. Furthermore, it will fail to identify that what it is sensing is shifted away from the distribution of observation it saw during training. The diminishing potential of models in Out-of-distribution (OOD) scenarios is worrying to AD applications where the safety of the passengers is paramount. AD agents must recognise scenes which are OOD and recover from them safely and non-erratically.

To address these challenges and develop more robust and reliable autonomous driving systems, various techniques can be employed. Bayesian deep learning provides a principled framework for quantifying uncertainty, allowing models to express their confidence in their predictions [25]. Ensemble methods, which combine multiple models, can improve robustness and reduce uncertainty [26]. Adversarial training can enhance the model’s ability to handle adversarial attacks and OOD data [27]. By leverag-

ing these techniques, autonomous vehicles can be developed that are more capable of handling uncertainty, making safer and more reliable decisions.

2.5 Generative Models

A subset of machine learning models, known as generative models, focuses on learning the underlying distribution that represents the dataset. By estimating the parameters of the dataset ψ , these models can generate new data samples that share the characteristics of the original data [28]. Due to the inherently probabilistic nature of generative models, the generated samples come with an associated probability which can be used to quantify uncertainty.

2.5.1 Variational Autoencoders

A Variational Autoencoder (VAE) is a generative model that extends the concept of AEs and are motivated by Bayesian probability theory [29]. VAEs draw samples for the latent space data point from some prior distribution, typically a multivariate normal.

$$z \sim p(z)$$

The sampled z gets sent to a decoder to estimate the parameter values ψ that describe the data $\mathbf{x}$. Sampling from this distribution gives us independent and identically distributed (i.i.d) samples for $\mathbf{x}$.

$$\mathbf{x}' \sim D(\psi)$$

where D is a parametric distribution.

A better latent space representation of the data will lead to better samples for z and thus allow the model to generate realistic $\mathbf{x}'$ s. What we want is to map each input data $\mathbf{x}^{(i)}$ to some posterior distribution $P(z|\mathbf{x}^{(i)})$ and draw samples of z from it.

$$z \sim p(z|\mathbf{x}^{(i)})$$

Using Bayes Theorem, rewriting the posterior as the following reveals a problem.

$$p(z|\mathbf{x}^{(i)}) = \frac{p(\mathbf{x}^{(i)}|z)p(z)}{p(\mathbf{x}^{(i)})}$$

Finding the marginal, $p(\mathbf{x}^{(i)})$, is intractable as it would require integrating the like-

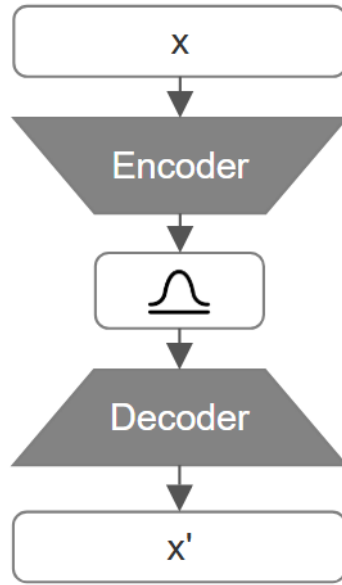


Figure 2.5: A Variational Autoencoder (VAE) model

likelihood multiplied by the prior for all values of z . Variational inference is a technique that defines a simpler distribution that approximates the real posterior while still being tractable. By sampling from this new distribution, the decoder can generate new data points that are similar to the training data.

$$z \sim q(z|x^{(i)})$$

This probabilistic approach eliminates the risk of overfitting to the data as the process is no longer deterministic. Secondly, it enables the generation of diverse and novel data samples, expanding the model's generative capabilities.

Chapter 3

Methodology

In this section, we discuss the methodology of testing the AD agent, specifically in OOD situations.

3.1 CARLA Simulator

3.1.1 CARLA

CARLA, an open-source program built on Unreal Engine, is a comprehensive simulation environment [30]. Equipped with LIDAR sensors, semantic segmentation capabilities, virtual towns with realistic road designs, traffic signals, pedestrians and other vehicles amongst others, CARLA provides a realistic testing ground for evaluating driving agents.

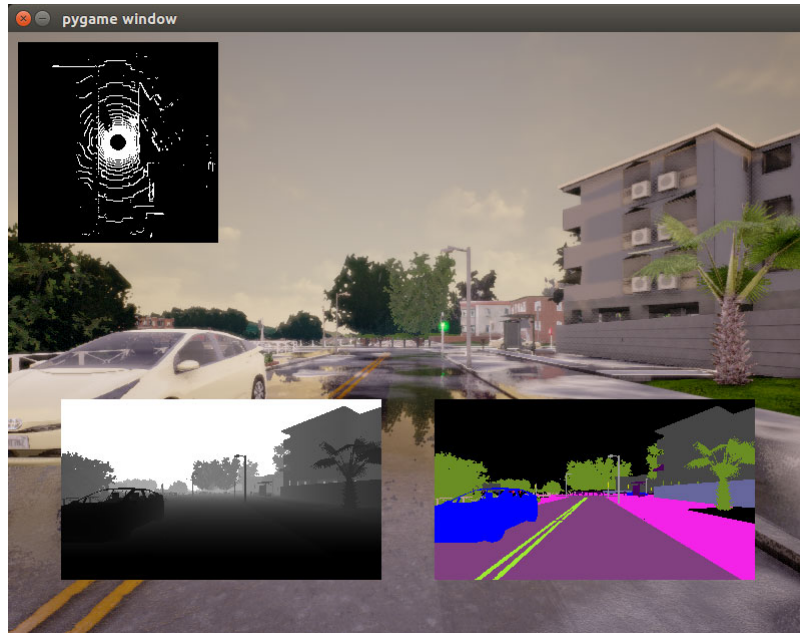


Figure 3.1: CARLA Simulator

The CARLA simulator follows a server-client architecture. The server is responsible for everything simulation-related: rendering, sensor measurements, physics computations, updating the environment states and actors, and much more. Interacting with its Application programming interface (API) allows access to sensor and traffic data and other relevant information that can be fed into our Pytorch-based DL models, which act as the client, for training and validation. Multiple clients can connect to a CARLA server at any given time.

3.1.2 CARNOVEL

CARNOVEL is a benchmark built on top of CARLA [31]. It provides challenging layouts of hills, sharp turns, and roundabouts for the agent to navigate through. These will be OOD scenes to what the agent will be trained on, making it a suitable platform to test the robustness of the model.

3.2 Dataset

3.2.1 CARLA Dataset

To train AD vehicles, we need samples of driving demonstrations. As we are testing our model in CARLA, we require CARLA-specific data points. AutopilotAgent is a trained agent based on CARLA’s own basic agent [32]. The AutopilotAgent drives around the simulation environment and aims to get from point A to B in the shortest distance possible, avoiding driving into other vehicles and following traffic rules while doing so. Demonstrations are gathered by collecting data from CARLA while AutopilotAgent is active in it. Table 3.1 shows the data collected and its description.

3.2.2 Data Sample Breakdown

A LIDAR sensor transmits beams which hit nearby objects and reflect back to the sensor. The angle and intensity at which the reflected signal is recorded paints a 3D picture of how the current environment looks like.

A velocity vector records the speed at which the agent is travelling in x , y and z directions.

The agent’s behaviour has to conform to traffic rules. Traffic lights in CARLA have a bounding box around them. `is_at_traffic_light` tells us if the agent is inside this boundary and `traffic_light_state` returns the current state of the traffic light (Red,

Table 3.1: Data sample breakdown

Observation	Description	Array Size
lidar	LIDAR point cloud array	(200,200,2)
velocity	The velocity vector for all three directions	(3,1)
is_at_traffic_light	Tells whether agent is near a traffic light	(1,1)
traffic_light_state	The current state of the traffic light	(1,1)
player_future	AutopilotAgent's trajectory	(4,2)

Yellow, Green). When we are away from the boundary then the state is either Off or Unknown.

The path the AutopilotAgent takes is recorded and saved under the player_future array. This records the (x,y) coordinate of the next four timesteps.

Together these observations form a single data sample. 200,000 of these samples form the train set and 40,000 samples complete the validation set.

3.3 Performance Metrics

Two key metrics are commonly used to assess the model's training performance: Average displacement error (ADE) and Final displacement error (FDE).

3.3.1 Average Displacement Error

The model, $q(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta})$, predicts a plan $\mathbf{y}$. ADE finds the error in the x and y coordinates between the model's predicted plan and the ground truth, $\mathbf{y}^*$ (player_future from Table 3.1), for each time step and averages it.

$$\text{ADE}(\mathbf{y}) = \frac{1}{T} \sum_{t=1}^T ||s_t^* - s_t|| \quad (3.1)$$

where $\mathbf{y} = (s_1, s_2, \dots, s_T)$ and $\mathbf{y}^* = (s_1^*, s_2^*, \dots, s_T^*)$. is the displacement at timestep t , going up to T timesteps.

Due to the stochastic nature of the model, the predicted trajectory will be different each time the model is called. minADE_k will return the smallest ADE evaluated for k

generated trajectories.

$$\min \text{ADE}_k(q) = \min_{\{\mathbf{y}^i\}_{i=1}^k \sim q} \text{ADE}(\mathbf{y}^i) \quad (3.2)$$

3.3.2 Final Displacement Error

FDE looks at the error at the final time step T , examining how far off we are from the destination. $\min \text{FDE}_k$ evaluates FDE from k trajectories.

$$\text{FDE}(\mathbf{y}) = ||s_T - s_T^*|| \quad (3.3)$$

$$\min \text{FDE}_k(q) = \min_{\{\mathbf{y}^i\}_{i=1}^k \sim q} \text{FDE}(\mathbf{y}^i) \quad (3.4)$$

3.3.3 CARNOVEL Metrics

Testing the model in CARNOVEL requires new benchmarks. There is no demonstration to compare to so ADE and FDE are no longer applicable. Agents are instead tested on whether they can consistently reach their destinations safely. Infractions per kilometre is a metric that counts how many road violations (changing lanes, hitting other vehicles etc) the agent commits per kilometre. This is a measure of how safe the agent is. Success rate examines how many times the agent was able to reach the destination without committing an infraction. This evaluates the consistency of the model.

Some state-of-the-art methods have already been tested in CARNOVEL, we will compare the agent's performance against those. The methods are Conditional Imitative Learning (CIL) [33], Deep Imitative Model (DIM) [34] and Robust Imitative Planning (RIP) [31].

CIL is a method that conditions its predictions on contextual information and high-level commands. DIM is a probabilistic predictive model of desirable behaviour able to plan interpretable expert-like trajectories to achieve specified goals. RIP is an extension of DIM. It utilises an ensemble of DIM models to choose trajectories that will help the agent recover from OOD scenarios. RIP is the only method that accounts for the epistemic uncertainty in its predictions.

Two primary approaches to implementing RIP are Model Averaging (MA) and the Worst-Case Method (WCM). Each model's output in the ensemble of models can be evaluated with a posterior probability of how likely it is to reach some goal location. This probability reflects the confidence in the model's prediction. In MA, the final

trajectory is computed as an average of the trajectories generated by each model weighted by its posterior. This approach leverages the strengths of multiple models and is more biased towards the more confident models. Inspired by robust control theory [35], WCM adopts a pessimistic approach to uncertainty. The agent assumes a worst-case scenario, considering the most challenging conditions that might arise. By planning for the worst case, the agent can make more conservative and safer decisions, reducing the risk of unexpected failures.

Chapter 4

Model Architecture

The block diagram in Figure 4.1 presents the general overview of how the agent interacts with the simulator. In this section, we look at the architecture of the AD agent in greater detail. We discuss the various tasks it's responsible for and justify methods chosen to perform those tasks.

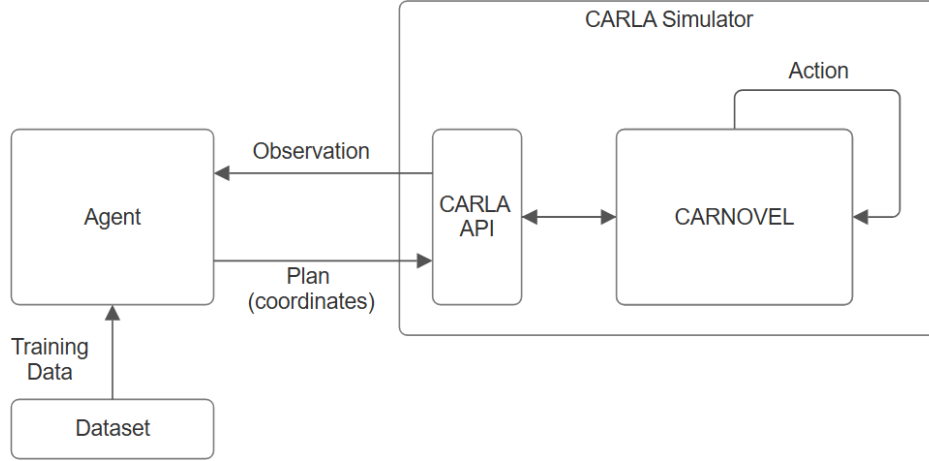


Figure 4.1: Overview

The agent is built on top of an expert model. The model is designed to receive observational inputs and generate trajectories. The agent decides what to do with those trajectories. The model, not the agent, is trained with the CARLA dataset. From now on, the model will be referred to as VAEModel and the agent as VAEAgent.

The VAEModel receives the observation data from the API. Firstly, the relevant features from the environment, conditioned on variables such as the vehicle's current velocity, need to be extracted from the observation. A VAE based generative model will take the encoded observation and compress it further into a latent vector while learning its distribution parameters. It will average the many samples drawn from the distribution and pass it to the network responsible for predicting a plan. The VAEAgent will optimise the plan further. The final plan gets sent to the simulator

through the API which updates the environment and passes new observations back to the model and the cycle is repeated. In this section, the components of the VAEModel are examined in detail.

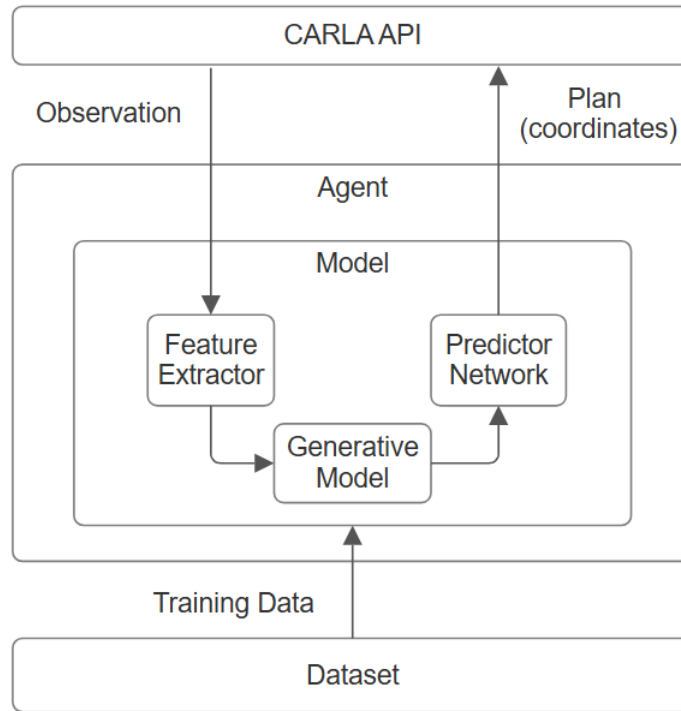


Figure 4.2: VAEModel Breakdown

4.1 Feature Extractor

The Feature Extractor performs two functions. First, it transforms the observation into something smaller by downsampling. Reducing the resolution makes it easier for the second function of the Feature Extractor which is to encode the scaled arrays into a single vector. The $(200, 200, 2)$ array of the LIDAR input gets converted to $(100, 100, 2)$.

A CNN architecture is chosen for encoding the scaled lidar array. The depth of the CNN model needs to be such that it is complex enough to capture intricate features and lightweight enough for offline deployment by devices with limited computational capabilities. The MobileNetV2 model was deemed suitable for this task. Its inverted residual structure, which utilises depthwise separable convolutions, keeps residual layers narrow rather than wide as seen in traditional residual models, allowing for efficient feature extraction while minimising computational cost.

4.1.1 MobileNetV2 Breakdown

Each line in Table 4.1 describes a layer, the operation performed on that layer and how many times that layer is repeated (**n**). **c** is the number of output channels and **s** is the stride size. A bottleneck layer performs depth-wise and point-wise convolution and residual addition. The final layer returns a 128-sized vector.

Table 4.1: MobileNetV2 Architecture

Input	Operator	c	n	s
$224^2 \times 3$	conv2d	32	1	2
$112^2 \times 32$	bottleneck	16	1	1
$112^2 \times 16$	bottleneck	24	2	2
$56^2 \times 24$	bottleneck	32	3	2
$28^2 \times 32$	bottleneck	64	4	2
$14^2 \times 64$	bottleneck	96	3	1
$14^2 \times 96$	bottleneck	160	3	2
$7^2 \times 160$	bottleneck	320	1	1
$7^2 \times 320$	conv2d	1280	1	1
$7^2 \times 1280$	avgpool	-	1	-
$1^2 \times 1280$	conv2d	128	-	-

Three additional vectors are concatenated to the Feature Extractor output and sent to the generative model to learn the distribution of the latent space. These are velocity, is_at_traffic_light, traffic_light_state from Table 3.1. These help provide context to the encoded observation, helping the agent make better decisions.

4.2 Generative Model

The 133-dimensional vector is further compressed into a 32-dimensional latent vector, denoted by z . To model the latent space, a multivariate Gaussian distribution is assumed, and two MLPs are employed to predict its mean and log variance. Each MLP consists of an input layer of 133 units, two hidden layers of 64 units and an output layer of 32 units. ReLU activation is applied to all layers of the mean predictor, except for the output layer, allowing for negative values. ReLU activation is also used on all log-variance predictor layers except the output layer, where a Softplus function

is applied. This ensures positive variance values and prevents numerical instability, particularly when dealing with small variances. By mapping the variance to a larger range, the model can learn more nuanced representations of the data.

Summing the outputs of the MLPs, which are deterministic, to obtain the latent vector z poses a problem. Since z is stochastic, it is undifferentiable and blocks the flow of gradients during backpropagation, hindering the training process. To overcome this, the reparameterisation trick is employed.

The solution is to make randomness an input to the model, rather than a consequence of it. We introduce a stochastic node ϵ , a sample from a standard normal distribution. The product of this output and the variance vector is what gets summed with the mean to give z . Now z is deterministic and the gradients can pass through it and train the MLPs of this generative model.

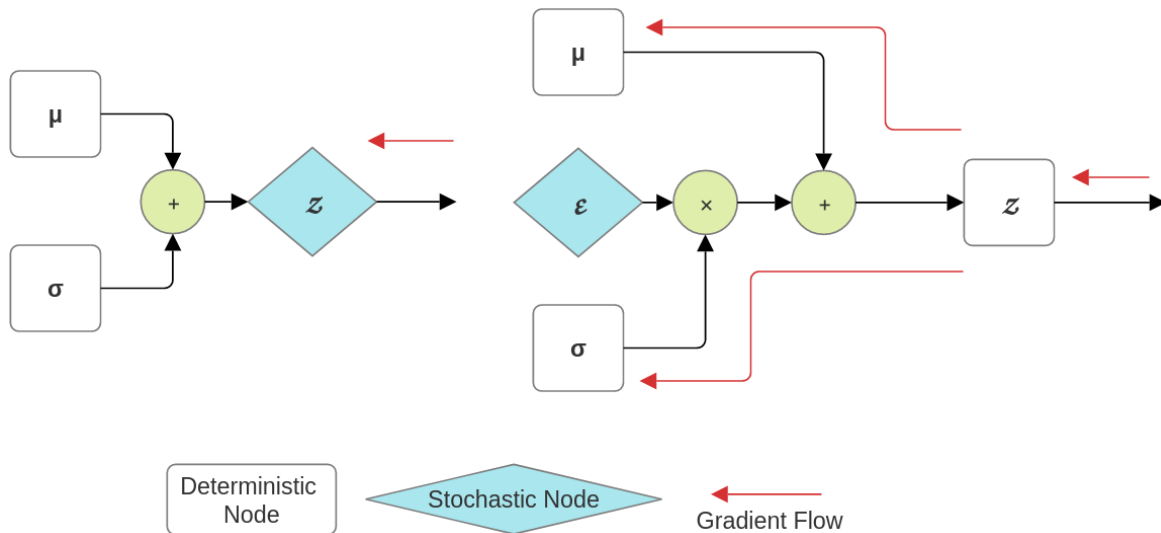


Figure 4.3: Reparameterisation Trick

As a result of employing the reparameterisation trick, multiple samples of the latent space can be drawn at will. An average of these samples will be what is sent to the decoder. Averaging them can reduce the variance in the latent space representation and introduce a form of regularisation. The hope is that it leads to better generalisation.

4.3 Predictor Network

The predictor network can be split into two parts, the GRU decoder and the estimator. They are illustrated in Figure 4.4 where the blue-shaded region is the GRU decoder part and the purple-shaded area is the estimator.

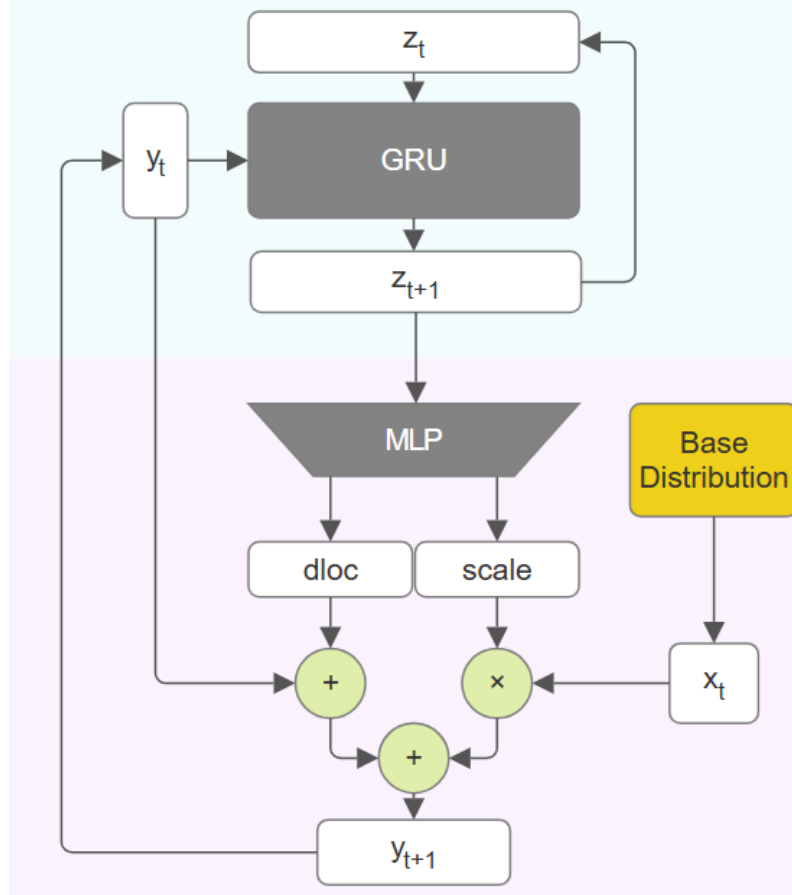


Figure 4.4: Predictor Network

4.3.1 GRU Decoder

The decoder iteratively predicts what the future latent space representation (z_{t+1}) of the observation will be, depending on the trajectory the AD agent follows ($\mathbf{y}$). Recurrent networks like GRUs are utilised as they are designed to capture long-term dependencies in sequential data. $\mathbf{y}$ is a tensor that stores (x, y) coordinates that the AD agent moves towards. We start by setting the initial state y_0 to $(0,0)$. This serves as the starting point for the trajectory generation process.

4.3.2 Estimator

The estimator part of the predictive network has an MLP that gets the updated latent vector z_{t+1} . Assuming that the change in position is a sample from a Multivariate normal (MVN) distribution, the MLP estimates the mean (dloc) and variance (scale) of this distribution and uses them to predict how much the AD agent moved to get to this new state. As dloc is the change in the mean, we add it to the current value of y_t and sum this with the scale multiplied by x_t , a sample from a simple zero mean and unit variance MVN called the base distribution.

$$x_t \sim \mathcal{N}(\boldsymbol{\mu}, \mathbf{I}) \quad (4.1)$$

where $\boldsymbol{\mu} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$, $\mathbf{I} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$. The base distribution has a probability distribution of

$$p_B(\mathbf{x}) = \frac{1}{2\pi|\mathbf{I}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \mathbf{I}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right) \quad (4.2)$$

x_t introduces stochasticity into the predictive process; sampling from a random distribution allows the model can explore a wider range of possible trajectories, leading to more diverse and creative outputs. The base distribution often serves as a simple, well-understood reference point. By transforming samples from this simple distribution, the model can generate complex and intricate trajectories.

$$y_{t+1} = (y_t + \text{dloc}) + \text{scale} \times x_t \quad (4.3)$$

Chapter 5

Training

To make the VAEModel an ‘expert’, it is trained on the CARLA Dataset discussed in Chapter 3. We need first to devise a loss function to train the VAEModel.

5.1 Training the Model

5.1.1 Negative Log Likelihood Loss

We train the VAEModel to ensure that the trajectory generated is a realistic one. We don’t want the predictor network to just throw out any random trajectory; it has to be appropriate to the context z . We can see from Figure 4.4 that x_t plays an important role in setting the value for y_{t+1} . If the VAEModel has learned well from the latent space, it should be able to produce a $\mathbf{y}$ such that if we rewrite Equation 4.3 as this

$$\hat{x}_t = \frac{y_{t+1} - (y_t + \text{dloc})}{\text{scale}} \quad (5.1)$$

the VAEModel can recover approximations of the sample obtained from the base distribution, $\hat{\mathbf{x}}$.

If the likelihood of $\hat{\mathbf{x}}$ being a sample from the base distribution is high, then we can be more confident in the trajectory $\mathbf{y}$ the network created. We can quantify this likelihood by evaluating the log probability of $\hat{\mathbf{x}}$ against the base distribution.

$$\log_prob = \sum_{i=0}^T \ln(p_B(\mathbf{x} = \hat{x}_t)) \quad (5.2)$$

As we are transforming our sample from one distribution to another, it is useful to track the scaling effect applied to the transformation. The determinant of the Jacobian measures how much the distribution was stretched or contracted. The spread of the distribution tells us how confident we can be about $\mathbf{y}$; a higher spread implies lower confidence.

When we perform supervised training, we already have the input and desired output

we want from the model. The VAEModel is trained to find parameters that are most likely to produce the desired output, meaning we are training under the maximum likelihood approach. The loss function thus has a Negative log likelihood (NLL) term. We can combine the two terms discussed to formulate an expression for NLL. This loss term will seek to simultaneously maximise the logprob (ensures the model learns well enough to perform inverse) and minimise the logabsdet (predict with the least uncertainty) terms.

$$\text{NLL} = -(\log_prob - \logabsdet) \quad (5.3)$$

5.1.2 Kullback Leibler Divergence Loss

The Kullback-Leibler (KL) divergence measures the difference between two probability distributions. In the context of VAEs, it measures the difference between the learned latent and prior distributions. By incorporating a KL loss term in the overall loss function, the model is forced to learn a distribution for the latent space that closely resembles the prior, typically a standard normal distribution.

$$\text{KL} = 0.5 \times \sum_{i=1}^D [1 + \log(\sigma_i^2) - \mu_i^2 - \sigma_i^2]$$

Beta is a hyperparameter that scales the influence of the KL loss in the loss function. A higher beta encourages more disentanglement in the latent space, which means each latent dimension captures a specific underlying feature in the data.

Combining the two loss terms allows us to complete the loss function.

$$\text{Loss} = \text{NLL} + (\beta \times \text{KL})$$

While setting a higher beta allows for more control over the sample to be generated, we initially want the VAEModel to learn the data distribution of the latent space. By starting with a β value of zero, the model is allowed to focus solely on minimising the NLL loss and producing the best trajectories. As we keep training the model, the beta coefficient is to increase gradually which will regularise the model and improve generalisation.

5.2 Further Optimisation

During inference, the AD agent encounters OOD situations where the trajectory generated by the VAEModel is not the most suitable; it is a good starting point but needs to be tweaked specifically for its situation. Another loss function is devised specifically for this case.

5.2.1 Prior

We start with a generated trajectory from the VAEModel. We use that $\mathbf{y}$ and send it back to the Predictor Network to recover $\hat{\mathbf{x}}$. We get the NLL of that by the same steps as Equation 5.3 We can name this as our imitation prior as this is our predicted trajectory based on the training applied by learning from the AutopilotAgent.

5.2.2 Goal Likelihood

Included in the observations from the CARLA API is a set of waypoints called the goal. This is the target location we want the agent to drive to. The goal likelihood assesses the quality of a plan by measuring its proximity to the desired goal locations. This metric leverages a mixture of Gaussian distributions, where each distribution represents a potential goal.

The plan is evaluated against each of these components for each timestep, and the overall likelihood is calculated based on the mean of all the probabilities. A higher goal-likelihood indicates that the plan terminates closer to one of the specified goals, suggesting a more successful and desirable outcome.

5.2.3 Posterior

By incorporating the goal-likelihood together with the imitation prior, a posterior can be formed.

$$\text{posterior} = (\text{imitation_prior} + \text{goal_likelihood}) \quad (5.4)$$

As the parameters of VAEModel can no longer be changed after training, the only other factor that directly affects the final trajectory predicted by the VAEModel is the value of the sample drawn from the base distribution as seen in Equation 4.3.

The VAEAgent's task is to find the optimum values for $\mathbf{x}$ so that the y_{t+1} produced in Equation 4.3 maximises the posterior in Equation 5.4. It takes the posterior as a

loss function and iteratively updates $\mathbf{x}$.

Chapter 6

Results

6.1 Training Evaluation

The training was performed for 500 epochs and the model weights were saved for every 5 epochs, meaning there are 100 variations of the model weight.

The model was loaded with the weight to be tested and tasked with generating five trajectories based on observations from the validation dataset. ADE, minADE, FDE, and minFDE were found using these trajectories.

The β hyperparameter is initially set to 0. While a higher β allows for more control over the sample to be generated, we initially want the model to learn the data distribution of the latent space. By starting with a β value of zero, the model is allowed to focus solely on minimising the NLL loss and producing the best trajectories. As we keep training the model, the β coefficient is set to increase every 25 epochs, allowing 24 training cycles for the model to adjust to its updated loss function. This approach will regularise the model and improve generalisation.

β is set to increment by 0.05 each time, by the number of training epochs approaches 500, the β value will be tending to 1. Figure 6.1 shows the metrics evaluated at each saved epoch.

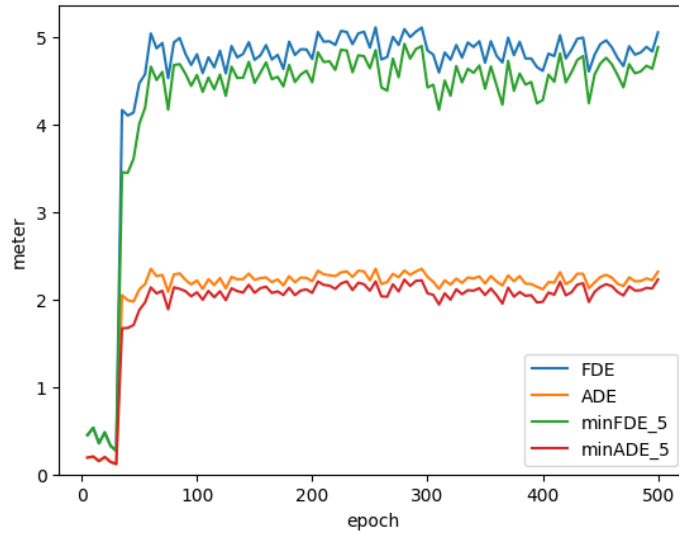


Figure 6.1: Model Evaluation (High Beta)

Signs that the model has learnt well is an overall trend of the metrics score falling. We can see that the first instance of the β value adjusting disrupts the learning and the model cannot recover anymore. The ADE and FDE are in the range of meters which is far from ideal. Furthermore, there is no indication of a falling trend in the values.

Clearly, the β jump was too high and the process needs revision. The training was performed again, this time by scaling β down by a magnitude. It will now go up in increments of 0.005. Figure 6.5 shows how the performance improved after making this change.

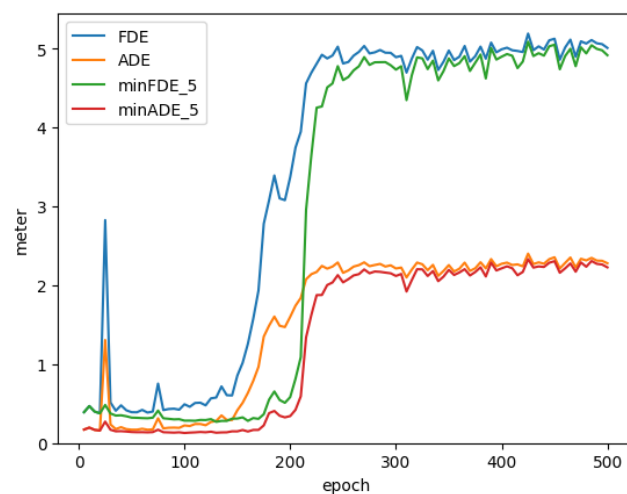


Figure 6.2: Model Evaluation (Low Beta)

Just like in the previous training cycle, the first time β changes causes a spike in the metric values. But this time, the model could recover from this setback and work towards the minima. The early stage of training exhibits the desired characteristics of the values going down. But eventually the performance worsens as β crosses some sweet spot and causes the ADE and FDE values to spike before settling in a similar range of values seen in Figure 6.1. The epoch number where all metrics are at their minimum is 65. The weights at that epoch will be used for all further tests. Table 6.1 shows the comparison of VAEModel with the weights from epoch 65 with a trained DIM model.

Table 6.1: Model Comparison

	ADE	minADE	FDE	minFDE
DIM	0.066	0.052	0.156	0.092
VAEModel	0.169	0.136	0.386	0.314

6.2 CARNOVEL

The agent is evaluated on a suite of OOD navigation tasks, a summary of which can be found in Appendix A The VAEAgent is compared to other imitation learning methods in the CARNOVEL benchmark. The outperforming method for each task is in bold.

Table 6.2: Comparison of different methods

Method	Abnormal Turns		Hills		Roundabouts	
	Success %	Infra/km	Success %	Infra/km	Success %	Infra/km
CIL	65.71	7.04	60.00	4.74	20.00	4.60
DIM	74.28	5.56	70.00	6.78	20.00	6.19
RIP (MA)	84.28	7.86	97.50	0.26	38.00	5.48
RIP (WCM)	87.28	4.91	87.50	1.83	42.00	4.32
VAEAgent	72.41	7.10	96.00	0.27	74.29	2.08

Every method is trained on the same AutopilotAgent demonstrations as our VAE-Model to ensure fairness. Each method was tested 10 times for every CARNOVEL task, with the simulator’s initial environment state being randomised for every new run.

6.2.1 Abnormal Turns

The VAEAgent is neither the best nor worst method when evaluated on the abnormal turns task. It betters the CIL method and is on par with DIM for successful navigation. They're all however overshadowed by both variants of RIP which are more than 10% more successful than any other technique. VAEAgent outdoes RIP (MA) in the infractions/km metric, but its DIM that drove with the fewest mistakes, making almost 1.5 infractions/km less than the next best, CIL.



Figure 6.3: Abnormal Turns

6.2.2 Hills

VAEAgent and RIP (MA) outperform every other in the Hills task. VAEAgent and RIP (MA) have near-identical results in both metrics, with RIP (MA) just edging out in front.



Figure 6.4: Hills

6.2.3 RoundABOUTs

VAEAgent achieves by far the best result compared to others in the roundabout task. It is the most successful in completing the assignment and commits less than half the mistakes as the next best alternative.

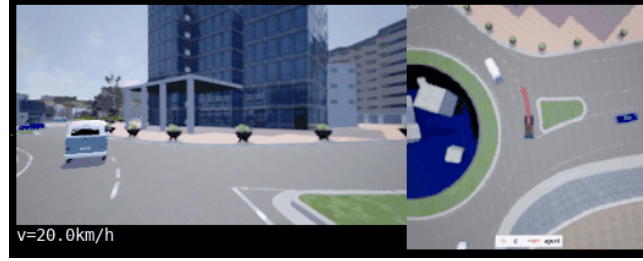


Figure 6.5: Roundabouts

6.3 Discussion

The results indicate that VAEAgent can safely navigate through the OOD situations. We can observe that the VAEAgent bettered DIM in two out of three test cases and matched it in the other, despite the model getting outperformed by DIM during training evaluation as per Table 6.1. This suggests that the model didn't overfit to the training data and was able to generalise better.

The most common source of mistake by VAEAgent in Abnormal Turns was lane inversions. The VAEAgent's errors weren't from lane inversions in Hills and Roundabouts but by colliding with other vehicles in the simulator.

The real test for VAEAgent is its performance against RIP, as they are both epistemic uncertainty aware models. VAEAgent was beaten in abnormal turns, matched RIP in hills but significantly outperformed it in the roundabouts task. VAEAgent does share the robustness quality present in RIP as it scored high in the success rate metric across all three tasks. Its strengths lie in driving on curved turns commonly found in the Hills and Roundabouts tasks and it excelled at it. The sharp corners in the Abnormal Turns tasks were more challenging for the VAEAgent to navigate.

The mixed results indicate that the different approaches of RIP and VAEAgent have complementary strengths and weaknesses. A hybrid approach that combines both methods' strengths could achieve even better performance. For instance, utilising an ensemble of models as RIP does and joining it with the generative properties in VAEAgent might be an avenue worth exploring.

Chapter 7

Conclusion

Implementing generative modelling, particularly with VAEs, within the context of trajectory planning in AD was the driving topic in this thesis. What was of particular interest was being aware of the inherent uncertainty of the model’s predictions and taking that into account when deciding the final path to take. The developed solution was trained on samples from a basic agent driving around the CARLA simulator.

By learning parameters to represent a latent space of the observable environment, new samples of the latent space can be generated. An average of these samples are passed to the decoder part of the model to formulate the path for the AD agent to take. Averaging these samples allows the agent to become more robust to novel situations.

The experimental results from Section 6.2 demonstrate the effectiveness of the proposed approach. The VAE-based method was able to match or outperform other imitation learning methods on the CARNOVEL benchmark, which particularly test the agent on scenarios that are OOD to its training. These findings highlight the potential of generative models to enhance the reliability of AD systems.

Appendix A

First Appendix



(a) AbnormalTurns0



(b) AbnormalTurns1



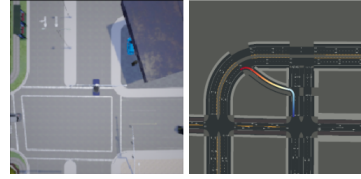
(c) AbnormalTurns2



(d) AbnormalTurns3



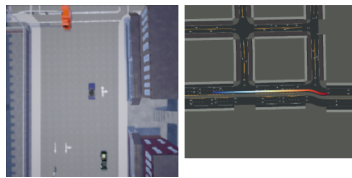
(e) AbnormalTurns4



(f) AbnormalTurns5



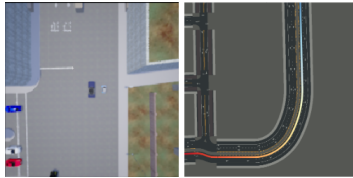
(g) AbnormalTurns6



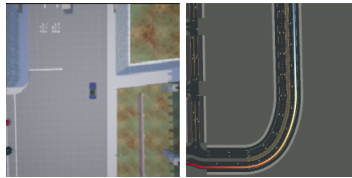
(h) Hills0



(i) Hills1



(j) Hills2



(k) Hills3



(l) Roundabouts0



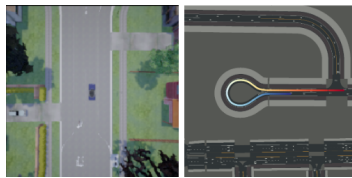
(m) Roundabouts1



(n) Roundabouts2



(o) Roundabouts3



(p) Roundabouts4

List of Figures

2.1	An MLP model	4
2.2	CNN model example [17]	5
2.3	GRU model [19]	6
2.4	An Autoencoder (AE) model	7
2.5	A Variational Autoencoder (VAE) model	10
3.1	CARLA Simulator	11
4.1	Overview	17
4.2	VAEModel Breakdown	18
4.3	Reparameterisation Trick	20
4.4	Predictor Network	21
6.1	Model Evaluation (High Beta)	28
6.2	Model Evaluation (Low Beta)	28
6.3	Abnormal Turns	30
6.4	Hills	30
6.5	Roundabouts	31

List of Tables

3.1 Data sample breakdown 13

4.1 MobileNetV2 Architecture 19

6.1 Model Comparison 29

6.2 Comparison of different methods 29

Bibliography

- [1] M. A. Nielsen, *Neural networks and deep learning*. Determination press San Francisco, CA, USA, 2015, vol. 25.
- [2] M. A. Khan, H. E. Sayed, S. Malik, T. Zia, J. Khan, N. Alkaabi, and H. Ignatious, “Level-5 autonomous driving—are we there yet? a review of research literature,” *ACM Computing Surveys (CSUR)*, vol. 55, no. 2, pp. 1–38, 2022.
- [3] Y. Gal *et al.*, “Uncertainty in deep learning,” 2016.
- [4] D. Namiot and E. Ilyushin, “Generative models in machine learning,” *International Journal of Open Information Technologies*, vol. 10, no. 7, pp. 101–118, 2022.
- [5] H. Gao, Z. Wang, Y. Li, K. Long, M. Yang, and Y. Shen, “A survey for foundation models in autonomous driving,” *arXiv preprint arXiv:2402.01105*, 2024.
- [6] I. Barabas, A. Todoruț, N. Cordoș, and A. Molea, “Current challenges in autonomous driving,” in *IOP conference series: materials science and engineering*, vol. 252, no. 1. IOP Publishing, 2017, p. 012096.
- [7] N. Piaseczna, R. Doniec, S. Sieciński, K. Barańska, M. Jędrychowski, and M. Grzegorzek, “Driving reality vs. simulator: Data distinctions,” *Electronics*, vol. 13, no. 14, p. 2708, 2024.
- [8] S. F. Ahmed, M. S. B. Alam, M. Hassan, M. R. Rozbu, T. Ishtiak, N. Rafa, M. Mofijur, A. Shawkat Ali, and A. H. Gandomi, “Deep learning modelling techniques: current progress, applications, advantages, and challenges,” *Artificial Intelligence Review*, vol. 56, no. 11, pp. 13 521–13 617, 2023.
- [9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2015.
- [10] N. K. Chauhan and K. Singh, “A review on conventional machine learning vs deep learning,” in *2018 International conference on computing, power and communication technologies (GUCON)*. IEEE, 2018, pp. 347–352.

- [11] S.-i. Amari, “Backpropagation and stochastic gradient descent method,” *Neuro-computing*, vol. 5, no. 4-5, pp. 185–196, 1993.
- [12] G. Hinton, *Learning representations by back-propagating errors*. Nature Publishing Group, 1988.
- [13] S. J. Prince, *Understanding Deep Learning*. The MIT Press, 2023.
- [14] J. Wu, “Introduction to convolutional neural networks,” *National Key Lab for Novel Software Technology. Nanjing University. China*, vol. 5, no. 23, p. 495, 2017.
- [15] P. Gavrikov and J. Keuper, “Cnn filter db: An empirical investigation of trained convolutional filters,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 19 066–19 076.
- [16] R. Nirthika, S. Manivannan, A. Ramanan, and R. Wang, “Pooling in convolutional neural networks for medical image analysis: a survey and an empirical study,” *Neural Computing and Applications*, vol. 34, no. 7, pp. 5321–5347, 2022.
- [17] W. Haibo, “Mitosis detection in breast cancer pathology images by combining handcrafted and convolutional neural network features,” 2014.
- [18] L. R. Medsker, L. Jain *et al.*, “Recurrent neural networks,” *Design and Applications*, vol. 5, no. 64-67, p. 2, 2001.
- [19] A. Mulahuwaish, “Covidmis20: Covid-19 misinformation detection system on twitter tweets using deep learning models,” 2022.
- [20] G. Shen, Q. Tan, H. Zhang, P. Zeng, and J. Xu, “Deep learning with gated recurrent unit networks for financial sequence predictions,” *Procedia computer science*, vol. 131, pp. 895–903, 2018.
- [21] R. Abdulhammed, H. Musafer, A. Alessa, M. Faezipour, and A. Abuzneid, “Features dimensionality reduction approaches for machine learning based network intrusion detection,” *Electronics*, vol. 8, no. 3, p. 322, 2019.
- [22] W. H. L. Pinaya, S. Vieira, R. Garcia-Dias, and A. Mechelli, “Autoencoders,” in *Machine learning*. Elsevier, 2020, pp. 193–208.

- [23] D. Feng, L. Rosenbaum, and K. Dietmayer, “Towards safe autonomous driving: Capture uncertainty in the deep neural network for lidar 3d vehicle detection,” in *2018 21st international conference on intelligent transportation systems (ITSC)*. IEEE, 2018, pp. 3266–3273.
- [24] Z. Zhang, Q. Liu, Y. Li, K. Lin, and L. Li, “Safe reinforcement learning in autonomous driving with epistemic uncertainty estimation,” *IEEE Transactions on Intelligent Transportation Systems*, 2024.
- [25] M. Sun, T. Zhang, Y. Wang, G. Strbac, and C. Kang, “Using bayesian deep learning to capture uncertainty for residential net load forecasting,” *IEEE Transactions on Power Systems*, vol. 35, no. 1, pp. 188–201, 2019.
- [26] R. Rahaman *et al.*, “Uncertainty quantification and deep ensembles,” *Advances in neural information processing systems*, vol. 34, pp. 20 063–20 075, 2021.
- [27] J. Dong, X. Qu, Z. J. Wang, and Y.-S. Ong, “Enhancing adversarial robustness via uncertainty-aware distributional adversarial training,” *arXiv preprint arXiv:2411.02871*, 2024.
- [28] J. Tomczak, *Deep Generative Modeling*. Springer, 2022.
- [29] D. P. Kingma, M. Welling *et al.*, “An introduction to variational autoencoders,” *Foundations and Trends® in Machine Learning*, vol. 12, no. 4, pp. 307–392, 2019.
- [30] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, “Carla: An open urban driving simulator,” in *Conference on robot learning*. PMLR, 2017, pp. 1–16.
- [31] A. Filos, P. Tigkas, R. McAllister, N. Rhinehart, S. Levine, and Y. Gal, “Can autonomous vehicles identify, recover from, and adapt to distribution shifts?” in *International Conference on Machine Learning*. PMLR, 2020, pp. 3145–3153.
- [32] CARLA, “Carla agents,” 2024. [Online]. Available: https://carla.readthedocs.io/en/0.9.12/adv_agents/
- [33] F. Codevilla, M. Müller, A. López, V. Koltun, and A. Dosovitskiy, “End-to-end driving via conditional imitation learning,” in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 4693–4700.

- [34] N. Rhinehart, R. McAllister, and S. Levine, “Deep imitative models for flexible inference, planning, and control,” *arXiv preprint arXiv:1810.06544*, 2018.
- [35] A. Wald, “Contributions to the theory of statistical estimation and testing hypotheses,” *The Annals of Mathematical Statistics*, vol. 10, no. 4, pp. 299–326, 1939.